the underlying URLs they're associated with. You can use URL generation to avoid littering your code with hardcoded URLs like `/product/view/3`. Instead, you can generate the URLs at runtime, based on the routing system. This approach makes changing the URL for a given endpoint easier: instead of your having to hunt down every place where you used the endpoint's URL, the URLs are updated for you automatically, with no other changes required.

By the end of this chapter, you should have a much clearer understanding of how an ASP.NET Core application works. You can think of routing as being the glue that ties the middleware pipeline to endpoints. With middleware, endpoints, and routing under your belt, you'll be writing web apps in no time!

6.1 What is routing?

Routing is the process of mapping an incoming request to a method that will handle it. You can use routing to control the URLs you expose in your application. You can also use routing to enable powerful features such as mapping multiple URLs to the same handler and automatically extracting data from a request's URL.

In chapter 4 you saw that an ASP.NET Core application contains a middleware pipeline, which defines the behavior of your application. Middleware is well suited to handling both cross-cutting concerns, such as logging and error handling, and narrowly focused requests, such as requests for images and CSS files.

To handle more complex application logic, you'll typically use the `EndpointMiddleware` at the end of your middleware pipeline. This